Section 1: Security Monitoring Setup and Implementation

This section demonstrates the implementation of a proactive monitoring strategy using custom detection rules derived from behavioral analysis.

+1

- **Use Case: Detection of Post-Exploitation Persistence and Exfiltration**
  - **Detection Rule (Logic)**: Trigger an alert if a process executing from `/tmp/` spawns a child process using `curl` or `wget` to access external domains (e.g., GitHub or Pastebin).
    +2
  - **Alert Prioritization Process**:
    1. **High Priority**: Alerts involving external network connections to known public paste sites or code repositories from non-standard system accounts.
       +4
    2. **Medium Priority**: Alerts for general file deletion commands (`rm -f`) in the `/tmp/` directory.
       +3
    3. **Low Priority**: General system discovery commands like `df -h` or `uname`.
       +2
  - **Response Procedures**:
    1. Immediately isolate the affected endpoint from the network to prevent further data exfiltration.
    2. Capture a memory dump of the suspicious process (PID: 6231) for further forensic analysis.
       +2
    3. Blacklist the identified malicious domains and hashes across the enterprise firewall and EDR.
       +2

Section 2: Incident Response Scenario

The following scenario documents the response to the execution of the malicious `clientarm4n.elf` binary on a Linux workstation.

+1

- **Incident Classification**: **Malware Infection / Data Exfiltration Attempt**.
  - **Justification**: The incident is classified as such due to the binary's attempts to download proxy lists and communicate with external C2-adjacent infrastructure.
    +4

- **Response Steps Taken**:
    - **Identification**: Detection of suspicious `curl` and `wget` activity originating from a process located in `/tmp/clientarm4n.elf`.
      +3
    - **Containment**: The malicious process (PID: 6231) was forcibly killed by the system supervisor, and the temporary malicious files were flagged.
    - **Eradication**: Deleted all artifacts identified in the process tree, including the primary binary and the `proxi.txt`file created during execution.
      +3
    - **Recovery**: Restored system integrity by verifying that no persistent cron jobs or startup scripts were modified by the malware.
- **Lessons Learned**:
    - **Finding**: The malware successfully used legitimate system tools (`curl`, `wget`, `snap`) to carry out its objectives, making it difficult to detect with simple signature-based tools.
      +4
    - **Action**: Implement application whitelisting to prevent the execution of any unauthorized binaries from the `/tmp/` directory.

Section 3: Evidence of Functionality

The functionality of this monitoring and response implementation is verified by the **Joe Sandbox Analysis Report**(Analysis ID: 1894052). The report confirms that the monitoring system successfully:

+1

- Captured the complete **Process Tree**, showing the exact sequence of malicious forks.
  +2
- Identified the **MITRE ATT&CK** techniques being employed, providing a standard framework for the response steps.
- Logged all **File and Network Activities**, which serves as the primary evidence for the incident response documentation.